Lab 3 – Virtualization with Containers (Docker)

Arquitectura de Xarxes – 2025-2026 3rd quarter

Universitat Pompeu Fabra

Introduction

In this lab, we explore **containerization** using Docker – one of the core technologies behind modern cloud infrastructure, used by AWS, Microsoft Azure, Google Cloud, and most production environments today.

In the theory sessions, we compared Virtual Machines (VMs) and containers: both provide isolation, but containers share the host operating system kernel and are significantly more lightweight, faster to start, and easier to distribute. This lab puts that theory into practice.

We will use **Nginx** as our example application. Nginx is a widely-used open-source web server – you have interacted with it every time you visited a website served from a Linux machine. It is a good choice for this lab: it requires no programming knowledge, its behaviour is easy to observe (it serves web pages and writes access logs), and it is ubiquitous in real deployments.

By the end of this lab, you will be able to:

- Build a custom Docker image from a Dockerfile
- Run and manage containers on a dedicated network
- Understand how container networking and port mapping work
- Persist data across container restarts using volume mounts
- Run multiple containers that communicate with each other
- Publish an image to a public registry (Docker Hub)

Note: use the same VM from previous sessions (one dedicated VM was assigned per group). Docker is already installed. Feel free to use your own Linux environment if you prefer.

Submission instructions

- Deliver a **PDF report** containing each question (Q1–Q22) followed by your answer and any relevant screenshots. Questions Q23–Q25 are optional.
- Include the names and NIAs of all group members on the cover page.
- Name the file: `AX_lab3_NIA1_NIA2_NIA3.pdf`
- Deadline: before the next lab session starts.
- **Academic integrity:** write your answers in your own words. AI-generated text is easy to spot. If we detect it, the grade is 0.

Lab Work

Step 1 – Verify Docker installation

Run the following commands to confirm Docker is installed:

```
Terminal -- verify installation

# Show available Docker arguments
sudo docker --help

# Check the installed version
sudo docker --version
sudo docker -v
```

Note: if Docker is not installed, follow the official documentation at `https://docs.docker.com/engine/install/`

Step 2 – Run Docker without sudo

By default, Docker requires `sudo`, which grants root-level privileges to every container and command. This is a security risk: if a malicious container escapes, it could compromise the entire host system.

A safer approach is adding your user to the `docker` group:

```
Terminal -- add user to docker group

# Add current user to the docker group
sudo usermod -aG docker $USER

# Reboot the VM to apply the change
sudo reboot
```

After the reboot:

```
Terminal -- verify Docker works without sudo

docker --version

# Run the hello-world container to verify end-to-end
docker run hello-world
```

The `hello-world` container is a minimal image that prints a confirmation message and exits. It verifies the full installation pipeline: daemon, image pull, and container execution.

Step 3 – Pull the Nginx image

Docker images are downloaded from a **registry** – a centralised repository of container images. The default registry is Docker Hub (`hub.docker.com`).

Pull the official Nginx image:

Terminal -- pull Nginx image

```
docker pull nginx
```

List the images available on your host:

Terminal -- list local images

```
docker images
```

Q1: What is the size of the Nginx image? What factors do you think influence the size of a Docker image? Is a larger image preferable or a smaller one? Justify your answer.

Step 4 – Write a Dockerfile for a custom Nginx image

Rather than using the official Nginx image directly, we will build a **custom image** that serves our own HTML page.

Create a working folder and move into it:

Terminal -- create working folder

```
mkdir ~/lab3-docker && cd ~/lab3-docker
```

Create a simple HTML file named `index.html`. You can use `nano` or `gedit`:

Terminal -- create index.html

```
nano index.html
# Press Ctrl+O to save, Ctrl+X to exit
```

Content of `index.html`:

index.html

```
<!DOCTYPE html>
<html>
  <head><title>AX Lab 3</title></head>
  <body>
    <h1>Hello from Docker!</h1>
    <p>This page is served by Nginx inside a container.</p>
  </body>
</html>
```

Now, **write the Dockerfile from scratch** without referring to the theory slides. Create a file named `Dockerfile` (no extension) in the same folder. Your image must satisfy these requirements:

- Base image: `nginx:latest`
- Copy `index.html` into the directory Nginx uses to serve static files: `/usr/share/nginx/html/`

- Expose port `80`

Once written, verify the file content:

```
Terminal -- inspect your Dockerfile

cat Dockerfile
```

Q2: What does each line of your Dockerfile do? If the build fails, describe what you found wrong and how you fixed it.

Step 5 – Build the image

Build your custom image using the name `my-nginx`:

```
Terminal -- build image

docker build -t my-nginx .
```

The `-t` flag sets the image name. The `.` tells Docker to look for the Dockerfile in the current directory. If you are unsure what a flag does, use `--help`:

```
Terminal -- explore flags

docker build --help
```

Verify the image was created:

```
Terminal -- list images

docker images
```

Q3: You now see both `nginx` (the official image from step 3) and `my-nginx` (your custom image). Why is `my-nginx` not significantly larger, even though you added a file to it? What are Docker image layers?

Step 6 – Create a dedicated network

Docker provides networking capabilities so containers can communicate with each other and with the outside world. Before creating our own network, let us inspect what Docker provides by default:

```
Terminal -- inspect default Docker networks

docker network ls
docker network inspect bridge
docker network inspect host
docker network inspect none
```

Q4–Q6:

Q4. What is the purpose of each of the three default networks (`bridge`, `host`, `none`)?

Q5. When you run a container without specifying `--net`, which network does Docker use by default?

Q6. What limitation does the default `bridge` network have compared to a user-defined network?

Now create a dedicated network for our lab:

```
Terminal -- create custom network

docker network create --subnet=172.18.0.0/16 dockerNet
docker network ls
```

Check the host network interfaces:

```
Terminal -- inspect host interfaces

ip a
```

You will notice a new interface with IP `172.18.0.1` has appeared. This is the gateway that connects your host to containers on `dockerNet`.

Note: you will also see a `docker0` interface. This is the default bridge interface created by Docker at installation time. We will not use it in this lab, as we want a dedicated and isolated network.

Step 7 – Run a container

Start a container from your `my-nginx` image:

```
Terminal -- run container

docker run --detach --rm \
  --publish 8080:80 \
  --name webserver \
  --net dockerNet \
  --ip 172.18.0.2 \
  my-nginx
```

Here is what each flag does:

Flag	Meaning
`--detach`	Run in the background
`--rm`	Remove container automatically when stopped
`--publish 8080:80`	Map port 8080 on the host to port 80 inside the container
`--name`	Assign a human-readable name to the container
`--net`	Connect to a specific Docker network
`--ip`	Assign a static IP within that network

Flag	Meaning

Verify the container is running:

```
Terminal -- list running containers

docker container ls
```

Try it: attempt to start a second container with the same IP (`172.18.0.2`). What happens? Also try an IP outside the subnet (e.g. `172.20.0.5`). What does this tell you about how Docker validates IP assignments?

Step 8 – Test the web server and inspect container networking

Verify Nginx is serving your custom page:

```
Terminal -- test web server

# Using the container IP directly
curl http://172.18.0.2:80

# Using the host port mapping
curl http://localhost:8080
```

Q7: Both commands reach the same container. Explain why. What role does the `--publish 8080:80` flag play?

Now enter the container and inspect its network configuration from the inside:

```
Terminal -- open shell inside container

docker exec -it webserver sh
```

Once inside, run:

```
Inside container

ip a
exit
```

Then run `ip a` again on the host.

Q8–Q10:

Q8. What do the flags `-i` and `-t` mean in `docker exec -it`?

Q9. Why is the network interface output different inside the container compared to the host?

Q10. Can the host and the container be considered separate network entities? Justify your answer.

Step 9 – Execute commands inside a container

You can run individual commands inside a running container without opening an interactive shell:

Terminal -- inject commands into container

```
docker exec webserver ls
docker exec webserver mkdir test
docker exec webserver ls
```

Q11: You created a directory inside the running container. What do you think happens to that directory when the container is stopped and a new one is started from the same image? Why?

Step 10 – Fetch and follow container logs

Nginx logs every HTTP request it receives. Fetch the logs of your running container:

Terminal -- fetch container logs

```
docker logs webserver
```

Now follow the logs in real time (use `docker logs --help` to find the right flag). Then, in a **separate terminal**, send a few requests:

Terminal 2 -- generate traffic

```
curl http://localhost:8080
curl http://localhost:8080
```

Q12: Add a screenshot of the live log output. What information does each log line contain?

Step 11 – Log persistence problem

Stop the container:

Terminal -- stop container

```
docker container stop webserver
```

Since we used `--rm`, the container is automatically deleted when stopped. Verify:

Terminal -- verify container is gone

```
docker container ls -a
```

Start a new container from the same image and check its logs:

Terminal -- start fresh container and check logs

```
docker run --detach --rm \
  --publish 8080:80 \
  --name webserver \
  --net dockerNet \
  --ip 172.18.0.2 \
  my-nginx

docker logs webserver
```

Q13: What happened to the logs from the previous container? Is this a problem in a real-world scenario? Describe a concrete case where losing logs would have serious consequences.

Step 12 – Persist logs with a volume mount

To solve the log persistence problem, we use a **volume mount**: a host directory is linked to a path inside the container, so data written there survives container restarts.

Stop the current container and start a new one with a volume:

Terminal -- run container with volume mount

```
docker container stop webserver

docker run --detach --rm \
  --publish 8080:80 \
  --name webserver \
  --net dockerNet \
  --ip 172.18.0.2 \
  --volume /tmp/nginx-logs:/var/log/nginx \
  my-nginx
```

The flag `--volume /tmp/nginx-logs:/var/log/nginx` maps the host directory `/tmp/nginx-logs` to the path where Nginx writes its access and error logs inside the container.

Generate some log entries:

Terminal -- generate traffic

```
curl http://localhost:8080
curl http://localhost:8080
```

Inspect the log files on the host:

Terminal -- inspect logs on host

```
ls /tmp/nginx-logs
cat /tmp/nginx-logs/access.log
```

Q14–Q16:

Q14. What does `access.log` contain?

Q15. Stop the container and start it again. Are the previous log entries still there? Why?

Q16. What problem does the volume mount solve? What would happen without it?

Step 13 – Two-container communication

In real deployments, applications are split across multiple containers that need to communicate. Let us simulate this by running a second container on the same network.

Terminal -- run second container

```
docker run --detach --rm \
  --publish 8081:80 \
  --name webserver2 \
  --net dockerNet \
  --ip 172.18.0.3 \
  my-nginx
```

Test connectivity from `webserver` to `webserver2`, both by IP and by container name:

Terminal -- test inter-container connectivity

```
# By IP address
docker exec webserver curl http://172.18.0.3:80

# By container name
docker exec webserver curl http://webserver2:80
```

Q17–Q20:

Q17. Does communication by IP work? Why?

Q18. Does communication by container name work? Why? Would this work on the default `bridge` network?

Q19. Stop `webserver2`. What happens when you try to reach it from `webserver`?

Q20. In a production environment, what mechanism would you use to avoid hardcoding container IPs?

Step 14 – Serve different content per container

Currently both containers serve identical content. Let us change the page served by `webserver2` using a volume mount, without rebuilding the image.

Create a second HTML file:

```
Terminal -- create second HTML page

echo "<h1>This is webserver2</h1>" > ~/lab3-docker/index2.html
```

Stop `webserver2` and restart it mounting the new file:

```
Terminal -- restart webserver2 with custom content

docker container stop webserver2

docker run --detach --rm \
  --publish 8081:80 \
  --name webserver2 \
  --net dockerNet \
  --ip 172.18.0.3 \
  --volume ~/lab3-docker/index2.html:/usr/share/nginx/html/index.html \
  my-nginx
```

Verify both containers serve different content:

```
Terminal -- verify different content

curl http://localhost:8080
curl http://localhost:8081
```

Q21: You changed the content served by `webserver2` without rebuilding the image. What does this tell you about the relationship between images and containers? What is the conceptual difference between a Docker *image* and a Docker *container*?

Step 15 – Publish your image to Docker Hub

Docker Hub is the default public registry where images are stored and shared. We will publish `my-nginx` so that others (including your teacher) can download and run it.

Create an account at `https://hub.docker.com`.

Then create a personal access token: go to **Account Settings > Personal Access Tokens**. Grant at least **Read and Write** permissions.

Note: using a token instead of your password is a common security practice. If the token is accidentally exposed, you can revoke it without losing access to your account.

Log in from the terminal:

```
Terminal -- log in to Docker Hub

docker login -u <your-username>
# Paste your token when prompted
```

Tag and push the image:

```
Terminal -- tag and push image

docker tag my-nginx <your-username>/my-nginx:latest
docker push <your-username>/my-nginx:latest
```

Note: it is not necessary to create the repository on Docker Hub beforehand. The push operation creates it automatically.

Verify the image is visible at `https://hub.docker.com/repositories/<your-username>`. Add a screenshot to your report.

Q22: The image is public by default, meaning anyone can download and run it. What are the security implications of this? In what cases would you use a private registry instead?

Step 16 (Optional) – Multi-container application with Docker Compose

So far, we have managed containers with individual `docker run` commands. In practice, multi-container applications are defined declaratively using **Docker Compose**, which describes the entire application stack in a single file.

Create a file named `docker-compose.yml` in `~/lab3-docker/`:

`docker-compose.yml`

```
services:
  web:
    image: my-nginx
    ports:
      - "8080:80"
    networks:
      appnet:
        ipv4_address: 172.19.0.2
    volumes:
      - /tmp/nginx-logs:/var/log/nginx

  web2:
    image: my-nginx
    ports:
      - "8081:80"
    networks:
      appnet:
        ipv4_address: 172.19.0.3

networks:
  appnet:
    driver: bridge
    ipam:
      config:
        - subnet: 172.19.0.0/16
```

Start the application:

Terminal -- start Compose application

```
docker compose up -d
docker compose ps
```

Q23–Q25 (Optional):

Q23. Can `web` reach `web2` by service name (e.g. `docker exec ... curl http://web2`)? Why?

Q24. Run `docker compose down`. What happens to the containers, the network, and the volume mount?

Q25. What is the main advantage of `docker-compose.yml` compared to running individual `docker run` commands?

Step 17 (Optional) – Clean up

To remove all containers, images, and networks and free up disk space:

```
Terminal -- clean up Docker environment

docker system prune -a
```

Warning: this removes all local images, including ones you may want to keep. Read the confirmation prompt carefully before proceeding.

Summary of key Docker commands

Command	Description
`docker pull <image>`	Download an image from a registry
`docker build -t <name> .`	Build an image from a Dockerfile
`docker images`	List local images
`docker run ...`	Create and start a container
`docker container ls`	List running containers
`docker container ls -a`	List all containers (including stopped)
`docker exec -it <name> sh`	Open a shell inside a running container
`docker logs <name>`	Fetch container logs
`docker logs -f <name>`	Follow container logs in real time
`docker container stop <name>`	Stop a running container
`docker network ls`	List Docker networks
`docker network inspect <name>`	Show network details
`docker system prune -a`	Remove all unused Docker resources